

# Où vivent les variables et le code en mémoire

Lorsqu'un programme s'exécute, le système d'exploitation lui fournit un espace d'adressage virtuel. Le programme a donc l'impression de disposer de sa propre mémoire, organisée en différentes grandes zones.

Cette organisation est un modèle simplifié : elle permet de comprendre où sont généralement placées les données d'un programme. L'organisation exacte peut varier selon le système d'exploitation, le compilateur et les optimisations utilisées.

## Le segment de code

Le segment de code, aussi appelé *text segment*, contient principalement les instructions en code machine produites par le compilateur à partir des fonctions du programme.

Par exemple :

```
int additionner(int a, int b)
{
    return a + b;
}
```

Après compilation, les instructions machine correspondant à cette fonction seront placées dans le segment de code de l'exécutable.

Cette zone est normalement marquée comme exécutable et généralement non modifiable.

## Le segment de données

Les variables ayant une durée de stockage statique existent pendant toute l'exécution du programme.

On y retrouve notamment :

les variables globales, les variables static globales, les variables static locales.

Exemple :

```
int compteur = 10;

void foo()
{
    static int nombreAppels = 0;
    nombreAppels++;
}
```

compteur et nombreAppels existent tous les deux pendant toute l'exécution du programme.

Même si nombreAppels est visible uniquement à l'intérieur de foo(), sa durée de vie n'est pas celle d'une variable locale ordinaire.

## La pile — stack

La pile (*stack*) est utilisée principalement pour gérer les appels de fonctions.

Elle contient généralement : les variables locales, certaines informations nécessaires au retour d'une fonction, certains paramètres et des données temporaires utilisées par le programme.

Exemple :

```
void foo(int valeur)
{
    int resultat = valeur * 2;
}
```

Dans le modèle classique, `valeur` et `resultat` appartiennent à la pile associée à l'appel de `foo()`.

Quand `foo()` se termine, l'espace utilisé pour cet appel peut immédiatement être réutilisé.

On dit donc que les variables locales ordinaires ont une durée de vie automatique.

```
void foo()
{
    int a = 10;
}
```

`a` existe seulement pendant l'exécution de `foo()`.

Il faut cependant éviter de comprendre cela comme une règle physique absolue. Avec les optimisations, le compilateur peut conserver certaines variables directement dans des registres du processeur plutôt que de leur réserver réellement une case dans la pile.

Pour comprendre le langage, on utilise néanmoins le modèle :

variable locale automatique → stack

## Le tas — heap

Le tas (*heap*) est une zone utilisée pour les allocations dynamiques.

En C++, on demande généralement cette mémoire avec `new`.

```
int* p = new int;
```

Il est important de distinguer deux choses ici.

La variable :

```
p
```

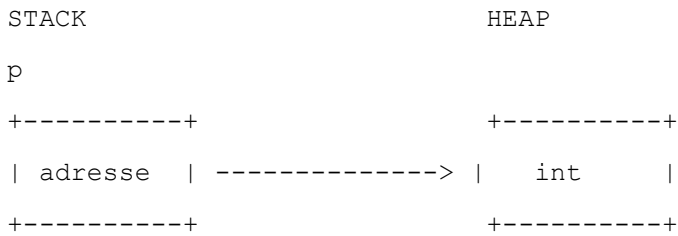
est elle-même une variable locale. Elle est donc généralement sur la stack.

Par contre, l'objet créé par :

```
new int
```

se trouve dans le heap.

On peut le représenter ainsi :

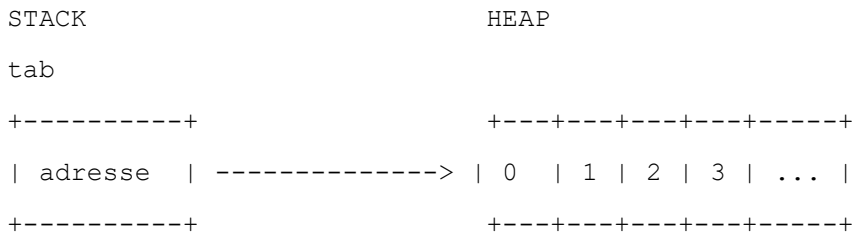


Le pointeur et l'objet pointé ne sont donc pas nécessairement dans la même zone mémoire.

Même chose avec un tableau dynamique :

```
int* tab = new int[100];
```

`tab` est généralement sur la stack, tandis que les 100 `int` sont réservés dans le heap.



Cette mémoire reste allouée jusqu'à ce qu'elle soit libérée :

```
delete p;
delete[] tab;
```

Si le programme perd l'adresse sans avoir effectué le `delete`, la mémoire ne peut plus être récupérée par le programme. On parle alors de fuite de mémoire (*memory leak*). Le système d'exploitation récupérera néanmoins la mémoire du processus lorsque celui-ci se terminera.

## Qui gère le heap ?

Lorsqu'on écrit :

```
int* p = new int[1000];
```

`new` ne demande pas nécessairement directement au système d'exploitation exactement  $1000 * \text{sizeof(int)}$  octets. Le programme utilise généralement un allocateur mémoire fourni par la bibliothèque d'exécution.

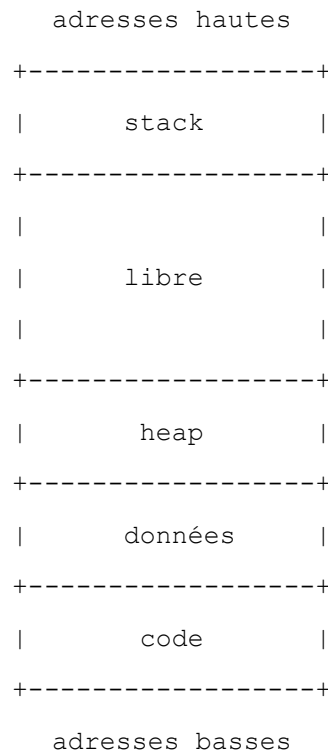
Cet allocateur gère une réserve de mémoire pour le processus. Lorsqu'il a besoin de davantage de mémoire, il peut demander au système d'exploitation de lui fournir de nouvelles pages de mémoire virtuelle.

Le programme travaille donc avec des **adresses virtuelles**. Il ne connaît généralement pas l'emplacement physique réel des données dans la mémoire RAM.

## Mémoire virtuelle et mémoire physique

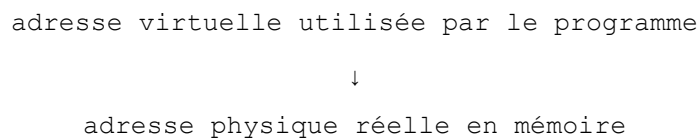
Il ne faut pas imaginer que la stack, le heap et les autres segments sont nécessairement placés les uns à côté des autres dans la RAM.

Une représentation comme celle-ci :



représente avant tout l'espace d'adressage virtuel du processus.

Physiquement, les différentes pages peuvent se trouver à des endroits complètement différents dans la RAM. Le système d'exploitation et le processeur, notamment grâce à la MMU (*Memory Management Unit*), effectuent la traduction entre :



Deux pages qui semblent voisines pour le programme ne sont donc pas nécessairement voisines physiquement.

[GeeksforGeeks "Memory Layout of C Programs"](#)

[Wikipedia « Data segment »](#)

[Wikipédia « Allocation de mémoire »](#)

[Scholarvox - Architecture et technologie des ordinateurs, Chapitre 12, section 5](#)